

PROJECT DOCUMENTATION REPORT

Full Stack E-Commerce Web Application using React, Spring Boot, MongoDB and Docker

Assignment No.	10	Date	11-April-2026
Name	S. Adithya	Reg. No.	3122245001007
Department	Computer Science and Engineering	Year / Sem	II Year / IV Sem
Subject Code	UCS3462	Subject Name	Internet Programming
Batch	2024–2028	Faculty	Dr . Prabavathy

Exercise Title

Develop a full stack e-commerce web application that allows users to register, log in, browse products, manage cart items, and provides an admin dashboard for product, user, and analytics management.

Objective

The objective of this project is to design and implement a microservices-based e-commerce application using modern web technologies. The application demonstrates:

- Frontend development using React with TypeScript and Vite.
- Backend service creation using Spring Boot.
- NoSQL data storage using MongoDB.
- REST API communication between frontend and backend services.
- Authentication and authorization using JWT.
- Container-based deployment using Docker Compose.

Problem Statement

The system is developed to simulate a simple online shopping platform. Users can create accounts, log in securely, view a dynamic product catalog, search and filter products, and add items to a cart. Administrators can manage products, review registered users, and view system analytics such as stock levels and projected revenue.

1 Software and Tools Used

Technology / Tool	Purpose
React + TypeScript + Vite	Frontend user interface development
Tailwind CSS	Styling and responsive design
Spring Boot	Backend REST microservices
MongoDB	Database for users, products, and carts
JWT	Authentication and role-based authorization
Docker Compose	Multi-service orchestration
Axios	API communication from frontend to backend
React Router	Frontend routing and protected navigation

2 Project Overview

This project is a microservices-based e-commerce application divided into frontend and backend modules. The main purpose of the system is to provide a simple but realistic online shopping workflow where users can create accounts, log in securely, browse products, apply filters, manage their cart, and where administrators can monitor the system through a dedicated dashboard.

The project was designed to combine modern frontend development with service-oriented backend development. On the frontend side, React with TypeScript is used to build an interactive user interface. On the backend side, Spring Boot microservices handle authentication, product operations, and cart management independently. MongoDB is used as the persistence layer, and Docker Compose simplifies local execution by starting the full stack together.

This structure makes the application easier to understand from an academic perspective because each module has a clear role and communicates through REST APIs. It also reflects the same architecture commonly followed in real-world web applications.

Frontend module:

- Login page
- Registration page
- Home page with live product catalog
- Search, category filter, sorting, and price filtering
- Cart management page
- Protected user routes
- Admin dashboard with overview, product management, user list, and analytics

Backend module:

- User Service on port 8081
- Product Service on port 8082
- Cart Service on port 8083
- MongoDB for persistent storage

3 Architecture Description

The application follows a distributed microservice architecture:

- The React frontend sends HTTP requests to backend services.
- The User Service handles registration, login, token validation, user listing, and user summary.
- The Product Service handles product CRUD operations, stock updates, and product summary generation.
- The Cart Service handles cart creation, item addition, quantity updates, item removal, and cart summary generation.
- The Cart Service communicates with the Product Service internally to calculate the total cart price.
- MongoDB stores separate collections/databases for user, product, and cart data.
- Docker Compose connects all services and provides a unified runtime setup.

3.1 System Architecture Flow

The complete system works in the following flow:

1. A user opens the frontend application in the browser.
2. The frontend sends authentication requests to the User Service for registration and login.
3. After successful login, a JWT token is generated and used to protect authenticated routes.
4. The frontend fetches product data from the Product Service and displays it in the catalog page.
5. When a user adds items to the cart, requests are sent to the Cart Service.
6. The Cart Service contacts the Product Service internally to calculate the latest total price based on product information.
7. The admin dashboard collects data from user, product, and cart endpoints to display summary cards and analytics.
8. MongoDB stores the required data for each microservice in separate databases.

3.2 Hosting Perspective

This project can be executed locally using Docker Compose, but it is also suitable for cloud deployment:

- The React frontend can be hosted on **Netlify**.
- The Spring Boot microservices can be hosted on **Render**.
- The MongoDB database can be connected through **MongoDB Atlas**.
- Production API URLs and secrets can be managed using environment variables.

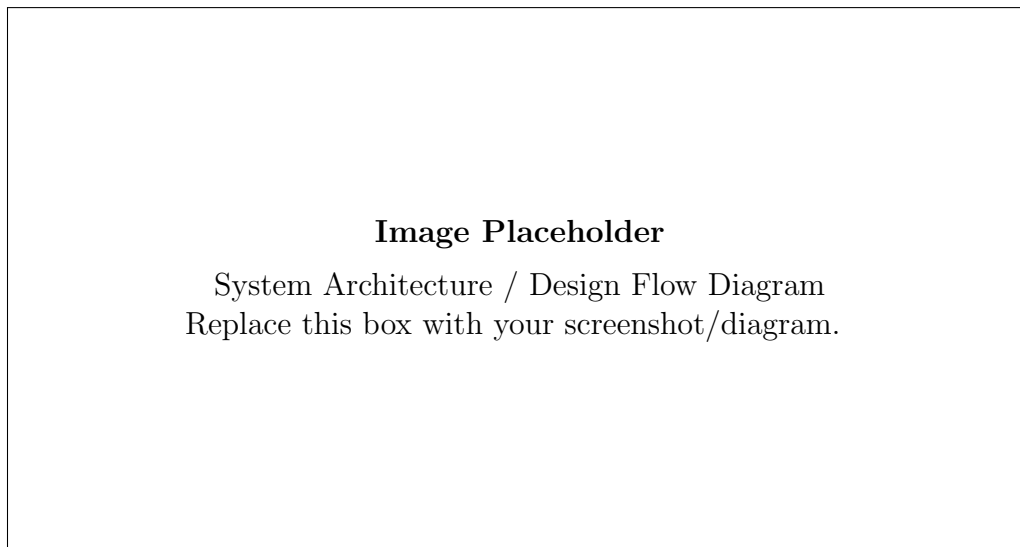


Figure 1: System Architecture / Design Flow Diagram

4 Program File Organization

Folder / File	Description
vite-project/src/App.tsx	Defines application routes for login, register, home, cart, and admin pages
vite-project/src/pages/Login.tsx	User login interface
vite-project/src/pages/Register.tsx	User registration interface
vite-project/src/pages/Home.tsx	Product listing, search, filter, and add-to-cart functionality
vite-project/src/pages/Cart.tsx	Cart display, quantity update, and remove item functionality
vite-project/src/pages/admin/*	Admin dashboard pages for overview, products, users, and analytics
backend/user-service/*	User registration, login, JWT validation, and summary service
backend/product-service/*	Product management and stock service
backend/cart-service/*	Cart management and analytics service
docker-compose.yml	Starts MongoDB, backend services, and frontend together

5 Important Frontend Files

Instead of printing the entire frontend source code, the following files are the most important for explaining the working of the application:

- **App.tsx** (`vite-project/src/App.tsx`)
Contains the main routing structure of the application. It connects all pages such as login, register, home, cart, and admin sections using React Router.
- **Home.tsx** (`vite-project/src/pages/Home.tsx`)
Implements the product listing page. It loads products from the backend and provides search, category filtering, price filtering, sorting, trending items, and add-to-cart operations.
- **Cart.tsx** (`vite-project/src/pages/Cart.tsx`)
Displays cart contents for the logged-in user and supports quantity updates, item removal, and total price display.
- **Login.tsx** (`vite-project/src/pages/Login.tsx`)
Accepts username and password, sends authentication requests, and redirects users based on their role.
- **Register.tsx** (`vite-project/src/pages/Register.tsx`)
Allows a new user to create an account through the registration endpoint.
- **ProtectedRoute.tsx** (`vite-project/src/components/ProtectedRoute.tsx`)
Protects authenticated routes and restricts admin-only access.
- **AuthContext.tsx** (`vite-project/src/context/AuthContext.tsx`)
Maintains login state and shares authentication details throughout the application.
- **Service files** (`vite-project/src/services/`)
Contains reusable Axios API methods for authentication, products, cart operations, and user summary requests.

6 Important Backend Files

The backend is organized into separate services. The following files are the most important for describing the implementation:

- **AuthController.java** (`backend/user-service/src/main/java/com/ecommerce/userservice/controller/AuthController.java`)
Handles login and token validation endpoints and returns JWT-based authentication responses.
- **UserController.java** (`backend/user-service/src/main/java/com/ecommerce/userservice/controller/UserController.java`)
Manages user registration, retrieval of users, admin listing, and user summary endpoints.
- **UserService.java** (`backend/user-service/src/main/java/com/ecommerce/userservice/service/UserService.java`)
Contains password encoding, login verification, role assignment, and user summary generation logic.
- **ProductController.java** (`backend/product-service/src/main/java/com/ecommerce/productservice/controller/ProductController.java`)
Exposes REST endpoints for adding, viewing, updating, deleting, and summarizing products.
- **ProductService.java** (`backend/product-service/src/main/java/com/ecommerce/productservice/service/ProductService.java`)
Performs product sanitization, stock management, and inventory summary calculation.
- **CartController.java** (`backend/cart-service/src/main/java/com/ecommerce/cartservice/controller/CartController.java`)
Handles cart item insertion, retrieval, update, removal, and cart analytics endpoints.
- **CartService.java** (`backend/cart-service/src/main/java/com/ecommerce/cartservice/service/CartService.java`)
Contains the main cart business logic and interacts with the Product Service to compute total cart price.
- **application.properties** (`backend/*/src/main/resources/application.properties`)
Stores configuration details such as service ports, MongoDB URI, CORS settings, JWT secret, and internal service URLs.
- **docker-compose.yml** (`docker-compose.yml`)
Defines and orchestrates the frontend, backend services, and MongoDB in a single setup.

7 Frontend Features Implemented

- User authentication screens for registration and login
- Route protection for authenticated users
- Role-based protection for admin pages
- Product browsing with dynamic data from backend
- Search by product name or description
- Category-based filtering
- Sort by featured, price ascending, price descending, and stock
- Add product to cart
- Update quantity and remove products from cart
- Responsive admin dashboard

8 Backend Features Implemented

8.1 User Service

- Register new user
- Encrypt password before storing
- Login verification
- JWT token generation
- JWT token validation
- Return user summary including admin and customer count

8.2 Product Service

- Add product
- View all products
- View single product by ID

- Update product
- Delete product
- Update stock
- Generate product summary including total products, stock units, low stock count, and inventory value

8.3 Cart Service

- Add item to cart
- Fetch cart by user ID
- Update cart item quantity
- Remove item from cart
- View all carts for admin
- Generate cart summary including total carts, active carts, items in carts, and projected revenue

9 Major API Endpoints

9.1 User Service Endpoints

Method	Endpoint	Purpose
POST	/users	Register a new user
GET	/users	Get all users
GET	/users/admin/all	Get all users for admin view
GET	/users/summary	Get user statistics
POST	/auth/login	Authenticate user and issue JWT
GET	/auth/validate	Validate JWT token

9.2 Product Service Endpoints

Method	Endpoint	Purpose
POST	/products	Add a product
GET	/products	Get all products

GET	/products/{id}	Get product by ID
PUT	/products/{id}	Update product details
PUT	/products/{id}/stock	Update product stock
DELETE	/products/{id}	Delete product
GET	/products/admin/summary	Get product summary

9.3 Cart Service Endpoints

Method	Endpoint	Purpose
POST	/cart/{userId}/items	Add item to cart
GET	/cart/{userId}	Get cart for a user
PUT	/cart/{userId}/items/{productId}	Update cart item quantity
DELETE	/cart/{userId}/items/{productId}	Remove item from cart
GET	/cart/admin/all	Get all carts
GET	/cart/admin/summary	Get cart analytics summary

10 Database Details

MongoDB is used as the backend database. Separate databases are configured for each service:

- `user_db` for user account data
- `product_db` for product catalog data
- `cart_db` for cart and cart item data

Main collections / entities used:

- User
- Product
- Cart
- CartItem

11 Docker and Deployment Flow

The project contains a `docker-compose.yml` file that starts the following services together:

- MongoDB container
- User Service container
- Product Service container
- Cart Service container
- Frontend container

This setup simplifies project execution, testing, and demonstration by allowing all modules to run as a single integrated system. It removes the need to start each service manually and ensures that dependencies between MongoDB and the Spring Boot services are preserved correctly.

In a hosted environment, the same project can be separated into deployment layers. The frontend can be deployed through Netlify for static hosting, while the backend microservices can be deployed individually on Render. This model is useful because it keeps the frontend lightweight and allows backend services to scale independently. A managed MongoDB service such as MongoDB Atlas can be used in the cloud to make the application accessible from anywhere.

12 Observation

During project implementation and execution, the following observations were made:

- The microservices approach makes the application modular and easier to manage.
- JWT-based authentication protects secured routes and admin pages.
- The cart service depends on the product service to calculate accurate total prices.
- React and Tailwind CSS provide a responsive and user-friendly frontend experience.
- MongoDB is suitable for storing flexible JSON-like data structures used in this project.
- Docker Compose makes multi-service setup simpler and more consistent.

- Deployment using Netlify and Render is suitable because the application is naturally divided into frontend and backend layers.
- The admin dashboard improves maintainability by providing a clear summary of users, product stock, and projected cart revenue.

13 Sample Output Screens

The following screenshots represent the major interfaces of the application. Each page contains two images arranged one after another vertically for better readability in the printed report.

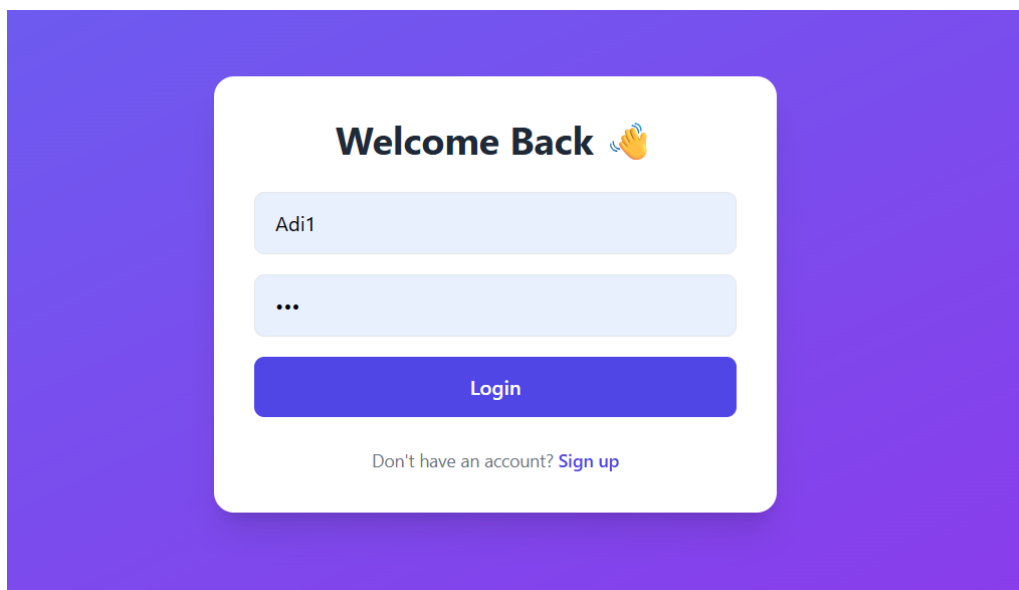
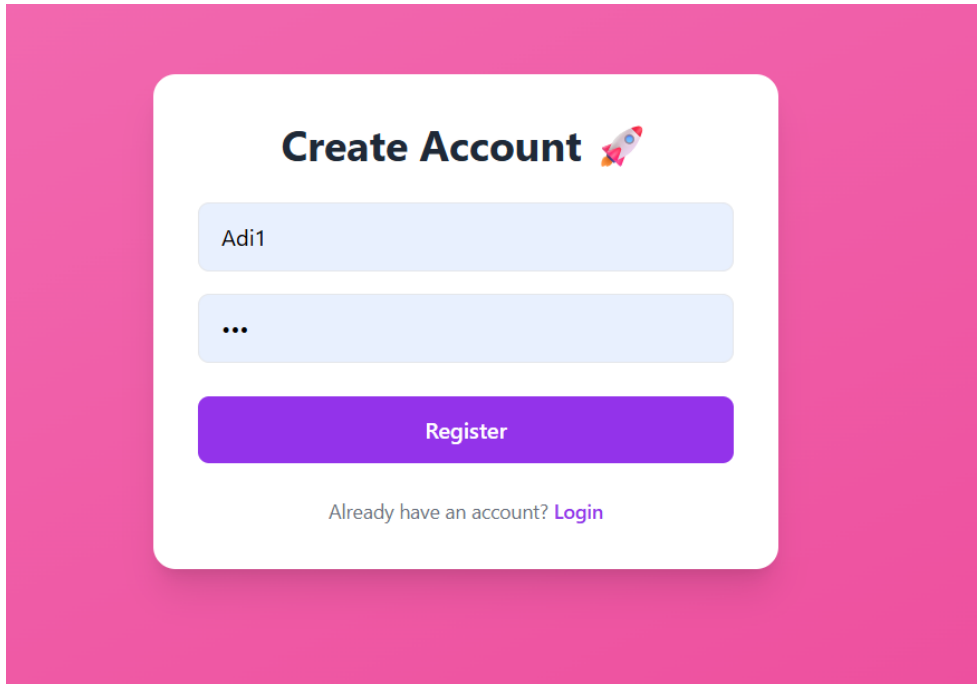


Figure 2: Login Page

A registration form titled "Create Account" with a rocket emoji. It features two light blue input fields: the first contains "Adi1" and the second contains three dots "...". Below these is a purple "Register" button. At the bottom, it says "Already have an account? Login" with "Login" as a link. The form is centered on a pink background.

Create Account 🚀

Adi1

...

Register

Already have an account? [Login](#)

Figure 3: Registration Page

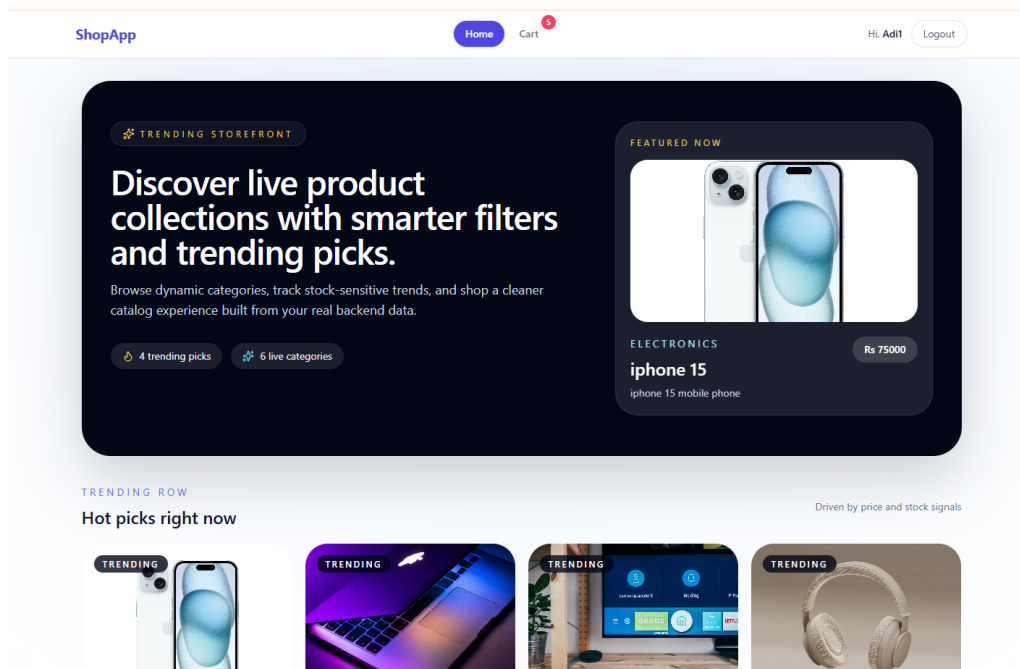


Figure 4: Product Catalog

Your Cart

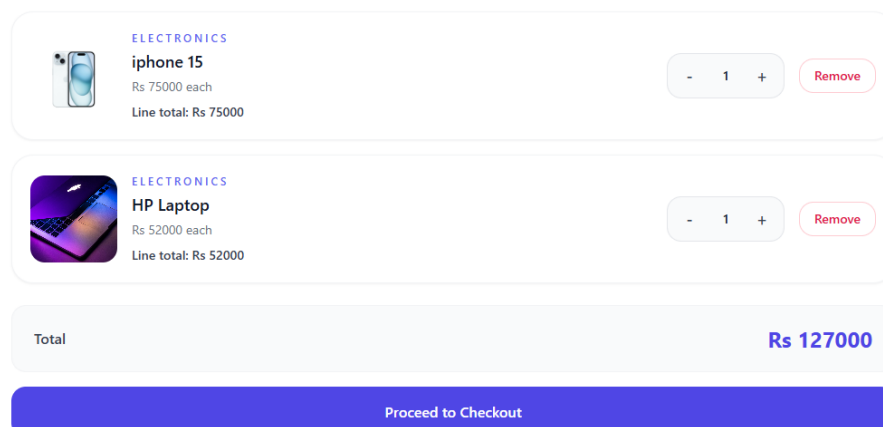


Figure 5: Shopping Cart

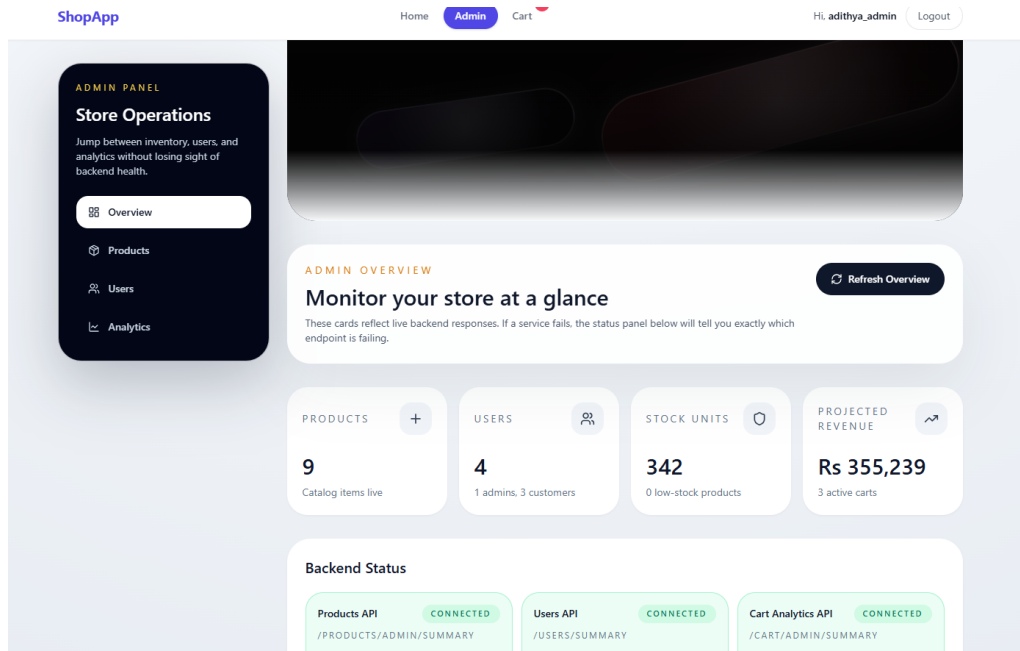


Figure 6: Admin Overview

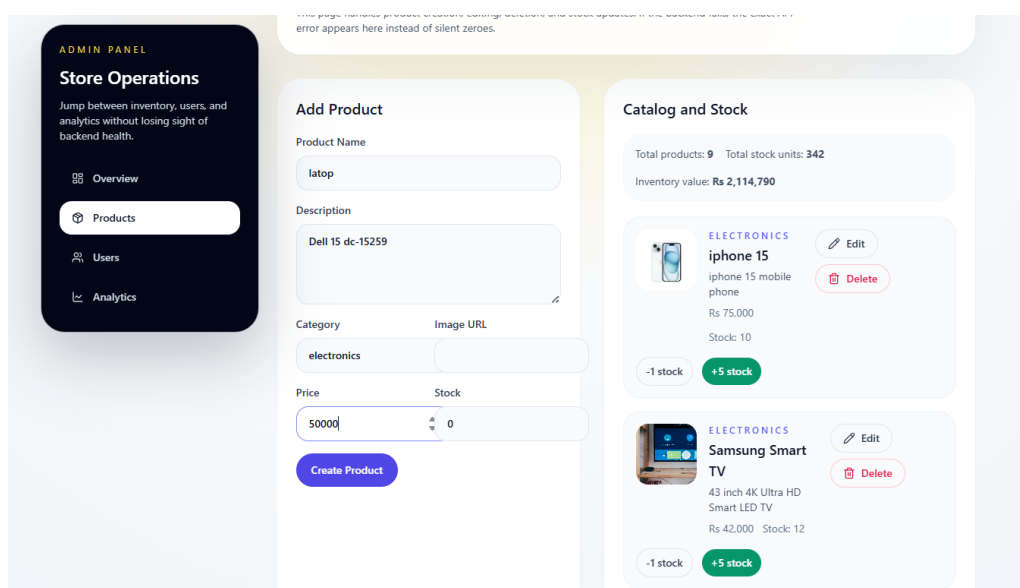


Figure 7: Product Management

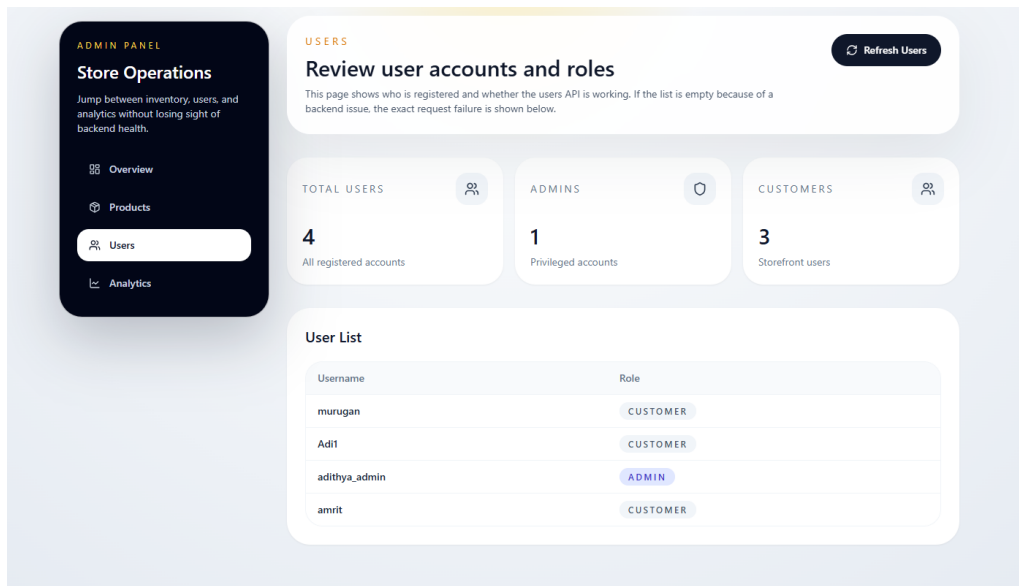


Figure 8: User Management

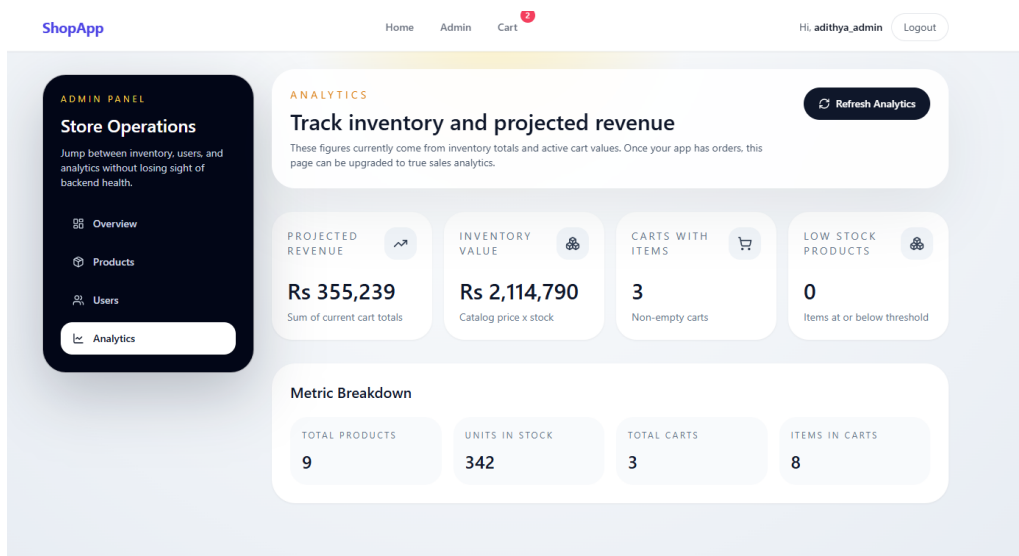


Figure 9: Analytics Dashboard

14 Result

Thus, the full stack e-commerce web application was successfully designed and implemented using React, Spring Boot, MongoDB, and Docker. The application supports authentication, product browsing, cart operations, and administrative monitoring through a structured microservices architecture.

15 Conclusion

This project demonstrates the practical implementation of modern internet programming concepts including frontend development, RESTful API design, backend microservices, NoSQL database integration, authentication, deployment planning, and containerized execution. The application successfully connects user management, product management, and cart processing into a single working e-commerce system. It also provides a strong foundation for future expansion such as payment integration, order tracking, production cloud deployment, and advanced reporting features.